# BÁO CÁO THỰC HÀNH

**PHÂN ĐOẠN NGỮ NGHĨA ẢNH GIAO THÔNG BẰNG U-NET TRÊN CAMVID**

Học phần: CO5085 – Deep Learning and Its Applications in Computer Vision

Sinh viên: Bùi Tiến Đức MSSV: 2680018

Ngày thực hành: 19/10/2026

**Nguồn** dataset : https://github.com/stonewst/CamVid-for-Segmentation

Trạng thái: Đã hoàn thành baseline U-Net sau 10 epochs và đánh giá lại checkpoint trên toàn bộ tập test.

## Tóm tắt kết quả

Baseline U-Net trên CamVid đạt test mIoU 0,3806 và Mean Dice 0,4469. Checkpoint tốt nhất được chọn tại epoch 9 theo validation mIoU. Báo cáo trình bày cấu hình, kết quả kiểm chứng và định hướng A2.1; UNet++ và phần tối ưu hóa chưa có kết quả thực nghiệm.

## 1. Mục tiêu và phạm vi

Xây dựng pipeline phân đoạn ngữ nghĩa ảnh theo nhãn từng pixel trên bộ dữ liệu CamVid: chuẩn bị dữ liệu, ánh xạ mask RGB sang class ID, huấn luyện U-Net và đánh giá bằng mean Intersection over Union (mIoU), Mean Dice. Nội dung này là thực hành Segmentation; A2.1 (chọn đề tài/bài báo) được trình bày riêng ở mục 9, không đồng nhất với A1 CNN–Transformer.

## 2. Môi trường thực nghiệm

| Thành phần | Cấu hình đã xác nhận |
|---|---|
| **Hệ điều hành** | Windows 11 |
| **Python** | 3.11 (.venv) |
| **PyTorch** | 2.11.0+cu128 |
| **GPU** | NVIDIA GeForce RTX 3050 Ti Laptop GPU (4 GB) |
| **CUDA PyTorch** | 12.8; torch.cuda.is_available() = True |

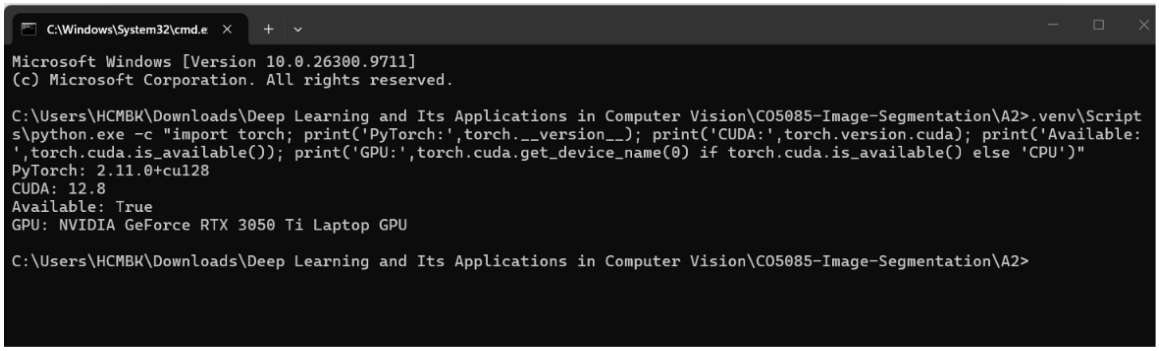
Môi trường đã ghi nhận bằng lệnh kiểm tra trong PowerShell tại thư mục A2:

```
.venv\Scripts\python.exe -c "import torch; print('PyTorch:',torch.__version__);
print('CUDA:',torch.version.cuda); print('Available:',torch.cuda.is_available());
print('GPU:',torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')"
```

Lệnh kiểm tra rút gọn:

```
.venv\Scripts\python.exe -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.is_available())"
```

**Hình 1. Xác nhận môi trường PyTorch và CUDA.**



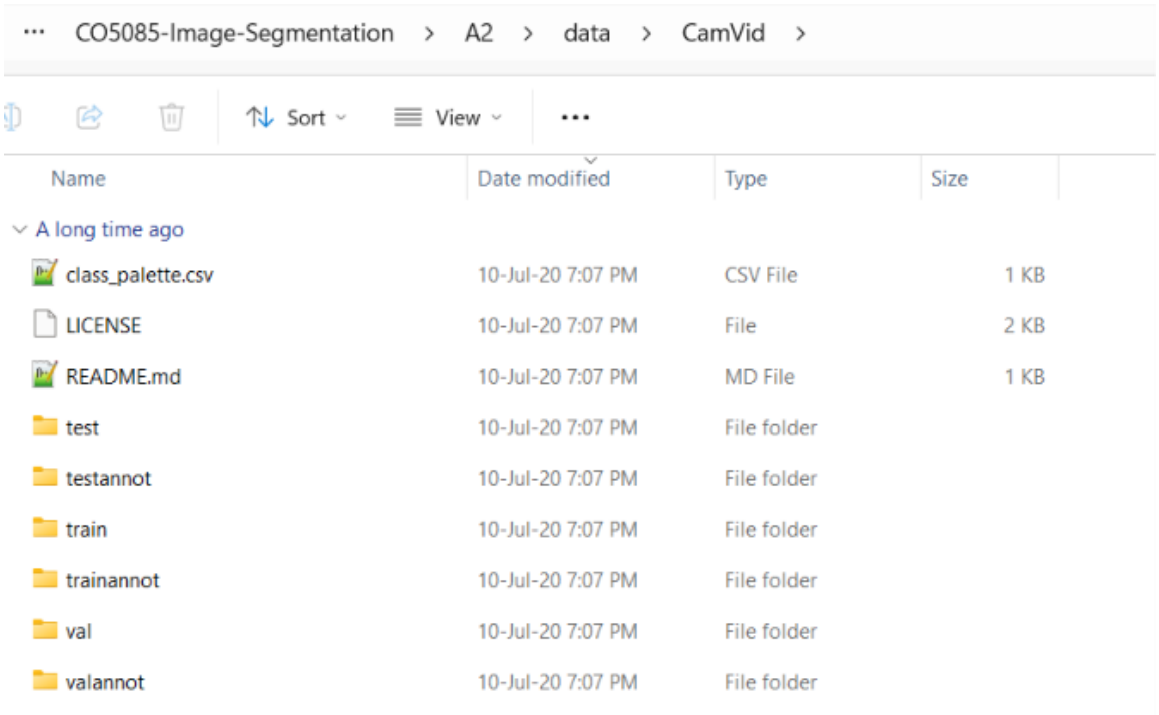

# 3. Bộ dữ liệu CamVid

Bộ dữ liệu gồm ảnh RGB cảnh đường phố và mask phân đoạn màu. Thực nghiệm sử dụng 11 lớp: Sky, Building, Pole, Road, Sidewalk, Tree, SignSymbol, Fence, Car, Pedestrian, Bicyclist. Vùng unlabeled được mã hóa thành 255 và loại khỏi loss/metrics.

| Tập | Số ảnh |
|---|---|
| Train | 367 |
| Validation | 101 |
| Test | 233 |
| Tổng | 701 |

```
dir data\CamVid
```

**Hình 2. Cấu trúc dữ liệu gồm train/trainannot, val/valannot, test/testannot.**

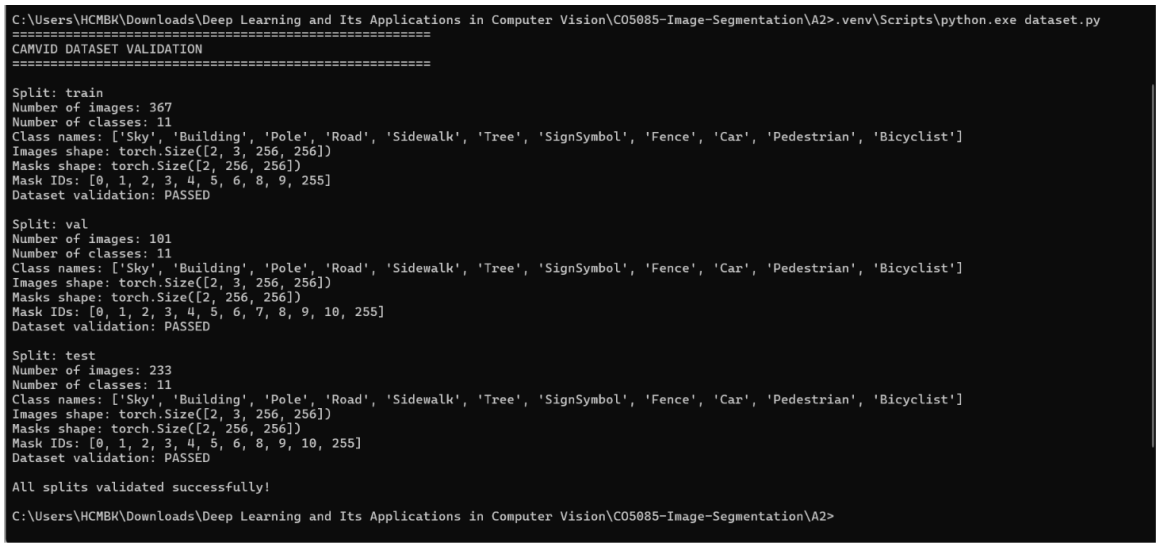


# 4. Tiền xử lý và kiểm tra dữ liệu

Ảnh được resize 256 × 256 bằng bilinear và chuẩn hóa về [0,1]. Mask được resize bằng nearest-neighbor để không tạo nhãn mới; ánh xạ RGB sang class ID 0–10. Mask có kích thước [B,H,W], kiểu long; vùng không gán nhãn = 255.

```
.venv\Scripts\python.exe dataset.py
```

**Hình 3. Kết quả kiểm tra ba tập dữ liệu: All splits validated successfully!**




Script show_camvid.py tạo ảnh minh họa một cặp ảnh và mask. Mã nguồn sử dụng:

```
from pathlib import Path
from PIL import Image
import matplotlib.pyplot as plt
root = Path("data/CamVid")
image_path = sorted((root / "val").glob("*.png"))[0]
mask_path = root / "valannot" / (image_path.stem + "_L.png")
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].imshow(Image.open(image_path).convert("RGB"))
ax[0].set_title("Input image")
ax[1].imshow(Image.open(mask_path).convert("RGB"))
ax[1].set_title("Ground truth RGB mask")
for a in ax: a.axis("off")
plt.tight_layout()
Path("results").mkdir(exist_ok=True)
plt.savefig("results/camvid_sample.png", dpi=180)
plt.show()
.venv\Scripts\python.exe show_camvid.py
```

**Hình 4. Ảnh đầu vào và mask nhãn thật CamVid (results/camvid_sample.png).**

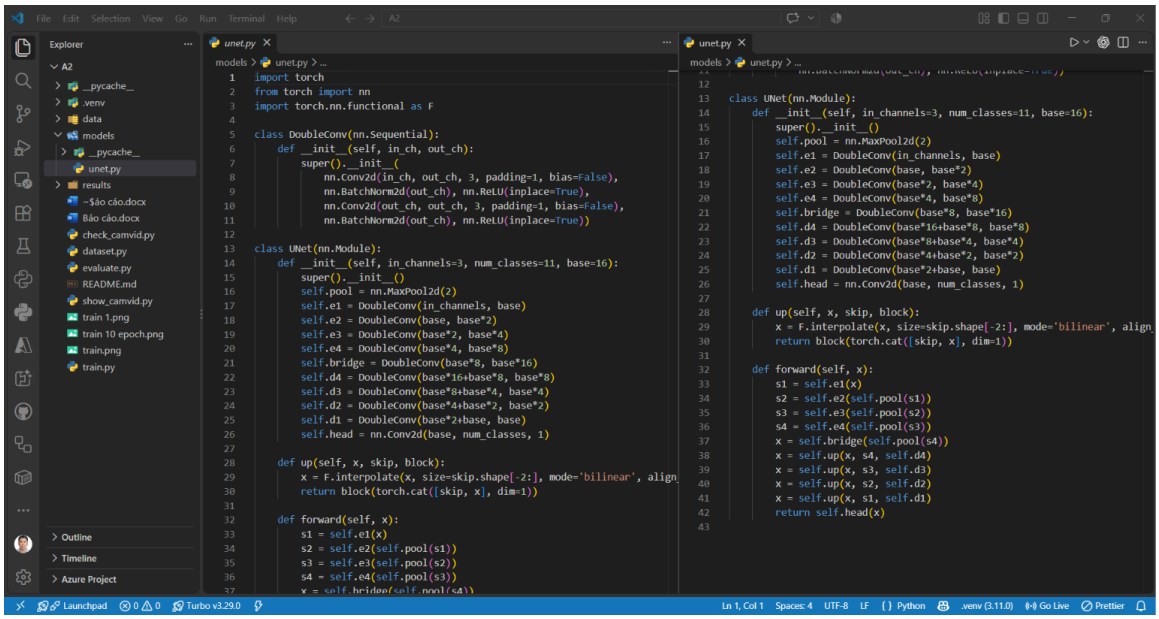



# 5. Kiến trúc và phương pháp

U-Net gồm encoder, bottleneck, decoder và skip connections. Đầu ra là logits cho 11 lớp tại mỗi pixel. Cấu hình base channels = 16. Huấn luyện dùng AdamW với learning rate 0,001 và CrossEntropyLoss(ignore_index=255); ảnh đầu vào 256 × 256, batch size 2. Mã hiện tại chưa sử dụng augmentation.

Mã kiến trúc được lưu tại models/unet.py; hình dưới là minh chứng cài đặt.

**Hình 5. Đoạn mã kiến trúc U-Net trong models/unet.py (ảnh chụp VS Code).**



```python
import torch
from torch import nn
import torch.nn.functional as F

class DoubleConv(nn.Sequential):
    def __init__(self, in_ch, out_ch):
        super().__init__(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True))

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=11, base=16):
        super().__init__()
        self.pool = nn.MaxPool2d(2)
        self.e1 = DoubleConv(in_channels, base)
        self.e2 = DoubleConv(base, base*2)
        self.e3 = DoubleConv(base*2, base*4)
        self.e4 = DoubleConv(base*4, base*8)
        self.bridge = DoubleConv(base*8, base*16)
        self.d4 = DoubleConv(base*16+base*8, base*8)
        self.d3 = DoubleConv(base*8+base*4, base*4)
        self.d2 = DoubleConv(base*4+base*2, base*2)
        self.d1 = DoubleConv(base*2+base, base)
        self.head = nn.Conv2d(base, num_classes, 1)

    def up(self, x, skip, block):
        x = F.interpolate(x, size=skip.shape[-2:], mode='bilinear', align
        return block(torch.cat([skip, x], dim=1))

    def forward(self, x):
        s1 = self.e1(x)
        s2 = self.e2(self.pool(s1))
        s3 = self.e3(self.pool(s2))
        s4 = self.e4(self.pool(s3))
        x = self.bridge(self.pool(s4))
        x = self.up(x, s4, self.d4)
        x = self.up(x, s3, self.d3)
        x = self.up(x, s2, self.d2)
        x = self.up(x, s1, self.d1)
        return self.head(x)
```

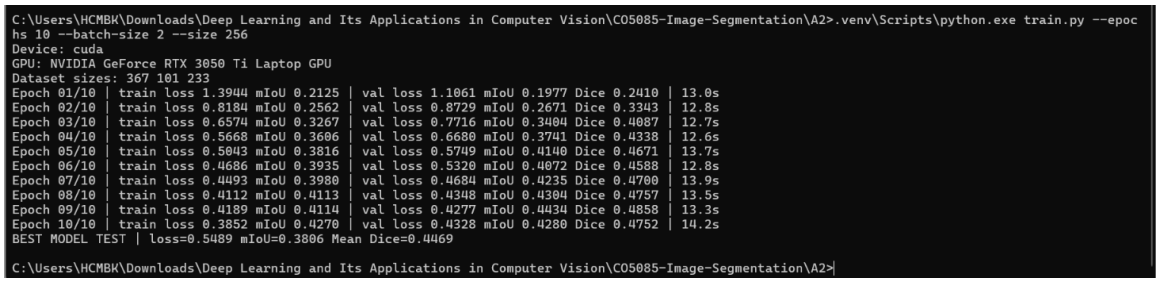
# 6. Huấn luyện U-Net trong 10 epochs

Thực nghiệm baseline đã hoàn thành 10 epochs. Lệnh dưới mô tả cấu hình đã sử dụng; không cần huấn luyện lại để đánh giá checkpoint. Nếu thực hiện thí nghiệm mới, cần lưu kết quả riêng để tránh ghi đè baseline.

```
.venv\Scripts\python.exe train.py --epochs 10 --batch-size 2 --size 256
```

Lịch sử được lưu trong results/history.csv. Validation mIoU cao nhất là 0,443417 tại epoch 9; epoch 10 đạt 0,428014. Checkpoint được chọn theo validation mIoU, không chọn theo kết quả test.

**Hình 6. Nhật ký huấn luyện U-Net trong 10 epochs và kết quả test.**




# 7. Đánh giá định lượng và biểu đồ

IoU = TP/(TP+FP+FN); Dice = 2TP/(2TP+FP+FN). Hàm scores trong train.py tính mIoU và Mean Dice từ confusion matrix cộng dồn trên toàn bộ tập, bỏ qua pixel nhãn 255 và lấy trung bình trên các lớp có union lớn hơn 0. Dice là chỉ số đánh giá, không phải hàm loss huấn luyện.

Bảng sau đối chiếu kết quả sau huấn luyện với lần nạp lại checkpoint. Chênh lệch tuyệt đối lớn nhất ở ba chỉ số test là 0,000004; hai lần đánh giá nhất quán ở độ chính xác 4 chữ số thập phân. Đánh giá độc lập dùng verify_checkpoint.py; evaluate.py chỉ tạo hình minh họa.

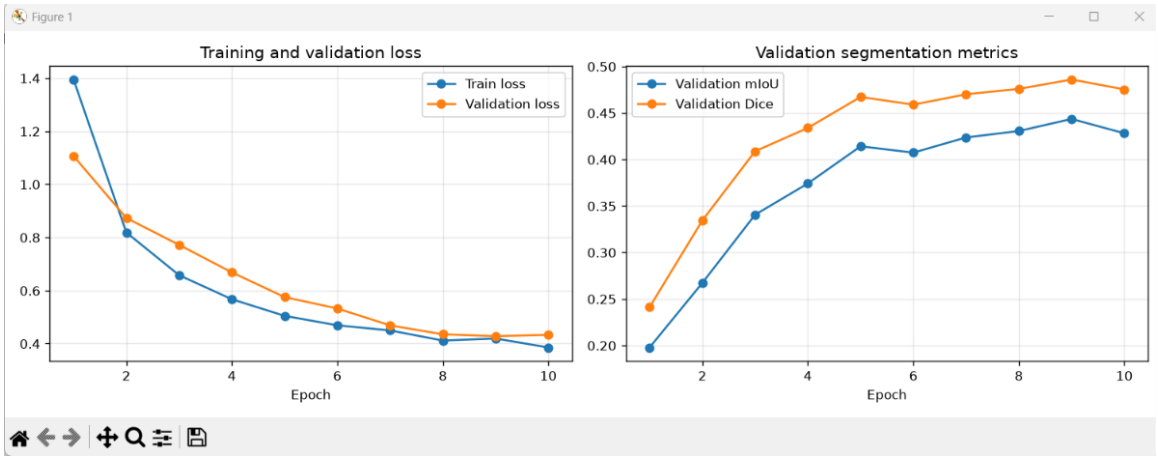
| Chỉ số | Sau huấn luyện | Kiểm chứng checkpoint |
|---|---|---|
| **Best validation mIoU** | 0,443417 (epoch 9) | Không tính lại validation |
| **Test loss** | 0,548880 | 0,548879 |
| **Test mIoU** | 0,380603 | 0,380599 |
| **Test Mean Dice** | 0,446893 | 0,446890 |

Biểu đồ được tạo từ results/history.csv bằng plot_history.py. Mã nguồn sử dụng:

```
import csv
from pathlib import Path
import matplotlib.pyplot as plt
path = Path("results/history.csv")
with path.open(encoding="utf-8-sig", newline="") as f:
    rows = list(csv.DictReader(f))
if not rows:
    raise ValueError("history.csv is empty")
print("CSV columns:", list(rows[0]))
def col(*names):
    for name in names:
        if name in rows[0]:
            return [float(r[name]) for r in rows]
    return None
x = col("epoch", "Epoch") or list(range(1, len(rows)+1))
series = [
    ("train_loss", "train_loss", "Train loss"),
    ("val_loss", "val_loss", "Validation loss"),
    ("val_miou", "val_miou", "Validation mIoU"),
    ("val_dice", "val_dice", "Validation Dice"),
```

```
]
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
found = False
for _, key, label in series[:2]:
    y = col(key, key.replace("_", " "))
    if y is not None:
        axes[0].plot(x, y, marker="o", label=label)
        found = True
for _, key, label in series[2:]:
    y = col(key, key.replace("_", " "))
    if y is not None:
        axes[1].plot(x, y, marker="o", label=label)
        found = True
if not found:
    raise ValueError("Update column names to match printed CSV columns")
for ax in axes:
    ax.set_xlabel("Epoch")
    ax.grid(True, alpha=0.3)
    ax.legend()
axes[0].set_title("Training and validation loss")
axes[1].set_title("Validation segmentation metrics")
plt.tight_layout()
plt.savefig("results/training_curves.png", dpi=180)
plt.show()
.venv\Scripts\python.exe plot_history.py
```

**Hình 7. Biểu đồ loss và chỉ số validation qua 10 epochs (results/training_curves.png).**

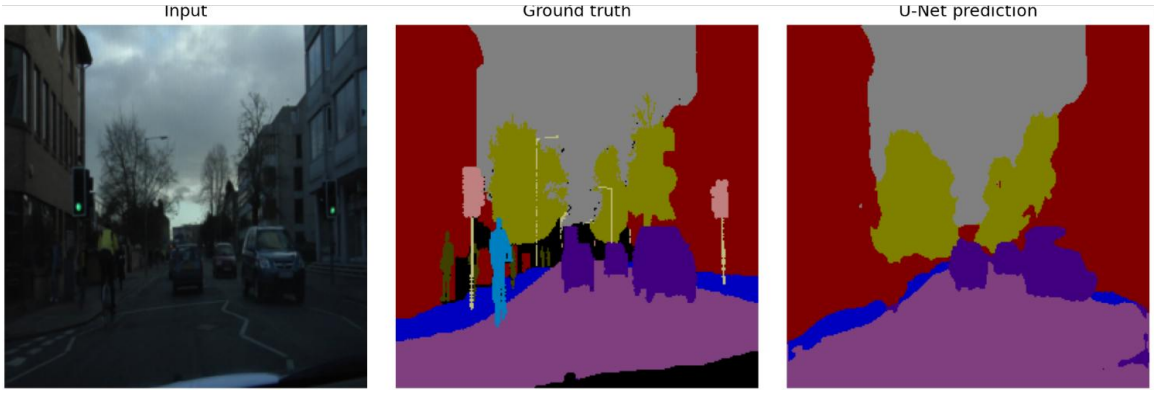



# 8. Trực quan hóa dự đoán

evaluate.py nạp checkpoint và tạo hình dự đoán cho ảnh test đầu tiên. Hình này dùng minh họa định tính; các chỉ số trong bảng được tính trên toàn bộ 233 ảnh test. Quan sát hình 8 cho thấy các vùng lớn như đường và bầu trời được tái hiện, nhưng biên đối tượng còn thô; vùng người đi bộ nhỏ bên trái không được giữ rõ như trong ground truth. Nhận xét này chỉ áp dụng cho ảnh minh họa.

```
.venv\Scripts\python.exe evaluate.py
start results\prediction.png
```

**Hình 8. So sánh ảnh gốc, ground truth và predicted mask (results/prediction.png).**




# 9. A2.1 – Đề xuất đề tài và bài báo

Đề tài dự kiến: So sánh U-Net và UNet++ cho phân đoạn ngữ nghĩa cảnh giao thông đô thị trên CamVid. Paper lựa chọn: Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh và J. Liang, “UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation,” IEEE Transactions on Medical Imaging, tập 39, số 6, trang 1856–1867, 2020. DOI: 10.1109/TMI.2019.2959609. Phạm vi là áp dụng kiến trúc lên CamVid, chưa phải tái lập đầy đủ thí nghiệm ảnh y sinh của paper gốc.

Câu hỏi nghiên cứu: UNet++ có cải thiện mIoU và Mean Dice so với U-Net dưới cùng cách chia dữ liệu và ngân sách huấn luyện hay không? Kế hoạch là giữ cùng kích thước ảnh, số epochs và quy trình đánh giá; có thể bổ sung số tham số, thời gian suy luận và mức sử dụng bộ nhớ. Hiện đã có baseline U-Net; chưa triển khai thí nghiệm UNet++ và phần tối ưu hóa.

# 10. Kết luận và giới hạn

Pipeline U-Net trên CamVid đã hoàn thành huấn luyện 10 epochs và đánh giá lại checkpoint. Test mIoU đạt 0,3806, Mean Dice đạt 0,4469 và test loss đạt 0,5489. Validation mIoU giảm từ 0,4434 tại epoch 9 xuống 0,4280 tại epoch 10, cho thấy kết quả có biến động; một lần chạy 10 epochs chưa đủ để kết luận quá khớp hoặc độ ổn định. Chưa có bảng IoU từng lớp hoặc nhiều lần chạy để đánh giá độ biến thiên. Kết quả hiện tại là baseline cho nghiên cứu tiếp theo.

# 11. Tái kiểm tra và đóng gói bài nộp

Huấn luyện và kiểm chứng: đã hoàn thành baseline 10 epochs và đánh giá lại checkpoint; kết quả lưu tại results/test_metrics_verified.txt.

Đánh giá độc lập tại A2: .\.venv\Scripts\python.exe .\verify_checkpoint.py. Lệnh này sử dụng checkpoint đã lưu, không huấn luyện lại.

Trực quan hóa: chạy evaluate.py, show_camvid.py và plot_history.py để tạo lại các hình từ dữ liệu và kết quả hiện tại.

Checkpoint để đánh giá: results/best_unet.pt. Cần cung cấp file này hoặc đường dẫn tải trong bộ nộp, cùng class_palette.csv và hướng dẫn chuẩn bị CamVid.

Thư viện dùng trong mã đã xem: torch, numpy, Pillow, matplotlib và tqdm. Lưu phiên bản môi trường bằng lệnh .\.venv\Scripts\python.exe -m pip freeze > requirements-lock.txt.

Đóng gói: mã nguồn gồm models/unet.py, README, lịch sử, metrics, hình, báo cáo và checkpoint hoặc link tải. Không đưa .venv và toàn bộ dataset vào repository.

Nếu học phần yêu cầu, ghi rõ phần sử dụng AI hỗ trợ và phần người thực hiện đã kiểm tra.

# Tài liệu tham khảo

1. Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.

2. Brostow, G. J., Fauqueur, J., & Cipolla, R. (2009). Semantic Object Classes in Video: A High-Definition Ground Truth Database. Pattern Recognition Letters.

3. Zhou, Z., et al. (2020). UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation. IEEE Transactions on Medical Imaging. DOI: 10.1109/TMI.2019.2959609.